

RAG · LLM ENGINEERING · LANGGRAPH

# RESEARCH RAG

A SELF-CORRECTING RAG SYSTEM THAT SEARCHES ARXIV  
—  
AND CHECKS ITS OWN WORK BEFORE ANSWERING.

LangChain

LangGraph

ChromaDB

GPT-4o-mini

Tavily

PyMuPDF

Built by [mpeshwe](#) · [namanmawandia](#)

# NAIVE RAG ANSWERS **CONFIDENTLY WRONG**

The standard recipe everyone ships:

- Embed the query
- Grab nearest chunks
- Generate an answer

It breaks the moment retrieval returns something **irrelevant** — the model answers anyway, with full

## THREE FAILURE MODES

- **Vocabulary mismatch** — one phrasing  $\neq$  all phrasings of an idea
- **No structure** — "papers under 50k words" can't be a similarity search
- **No fallback** — if the corpus lacks the answer, it hallucinates

# WHAT IS RESEARCH RAG?

A modular RAG system that intelligently searches and retrieves **arXiv research papers** using advanced query techniques + a corrective feedback loop.

## MULTI-ANGLE QUERIES

Generates 5 semantic variants + RAG Fusion, HyDE, Step-Back & Decomposition.

## STRUCTURED FILTERING

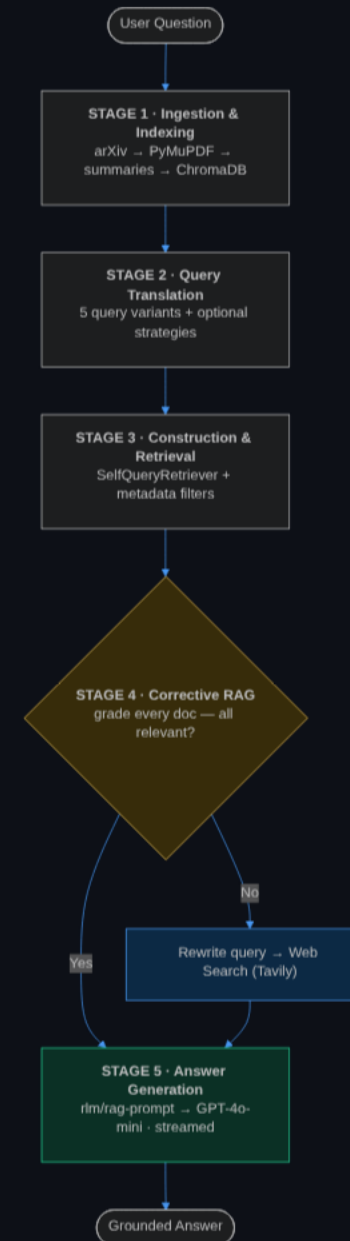
LLM turns plain English into metadata filters: year, authors, word count.

## SELF-CORRECTION

Grades its own retrievals; rewrites + web-searches when they fall short.

**5** stage pipeline · **6** RAG strategies · **1** LangGraph  
feedback loop

# THE 5-STAGE PIPELINE



Research RAG

# INGESTION & INDEXING

- **Download** papers from arXiv across 6 categories

cs.LG · stat.ML · cs.AI · cs.CL · cs.RO · cs.CR

- **Extract** full text + metadata via [PyMuPDF](#)
- **Summarize** each paper with GPT-4o-mini
- **Index** summaries into [ChromaDB](#)

## WHY CACHE?

Embedding + summarizing every paper is the most expensive step.

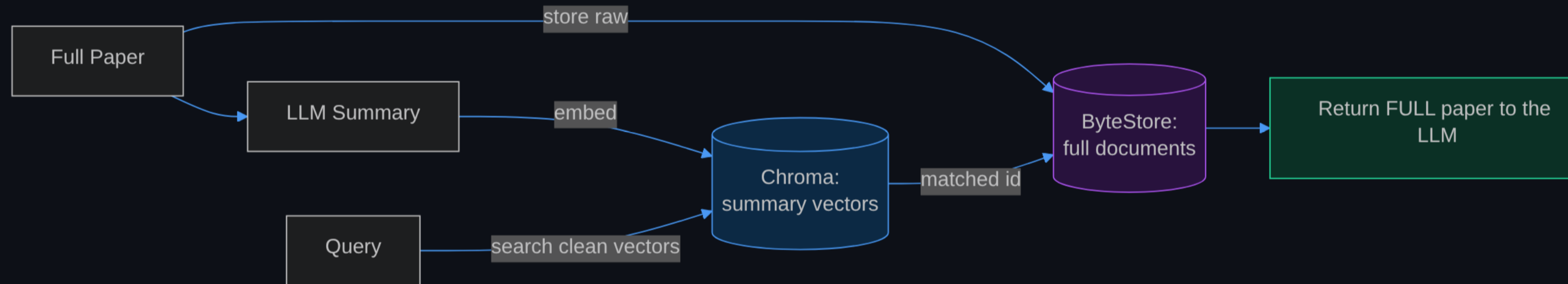
`documents_cache.pkl` and `summaries.pkl` mean you pay the OpenAI cost **once** — subsequent runs load instantly.

Files: `docDownload.py`, `indexing.py`

# SUMMARY-BASED INDEXING

## (MULTIVECTORRETRIEVER)

Embed something small & clean — return the full thing.



Best of both worlds: **precise retrieval** on tight summaries, **rich context** from full documents.



# QUERY TRANSLATION

One query is limiting. Generate **5 semantically diverse variants** to beat the vocabulary-mismatch problem — plus toolbox of strategies for harder questions.

| Technique                                 | Best for                                   |
|-------------------------------------------|--------------------------------------------|
| <b>Multi-Query</b> <small>default</small> | General use — 5 angles on one question     |
| <b>RAG Fusion (RRF)</b>                   | Re-ranking results across multiple queries |
| <b>Decomposition</b>                      | Complex, multi-part questions              |
| <b>Step-Back</b>                          | Abstract / conceptual questions            |
| <b>HyDE</b>                               | Sparse or highly technical corpora         |

File: QueryTranslation.py

Research RAG

# QUERY CONSTRUCTION & RETRIEVAL

The `SelfQueryRetriever` reads plain English and builds **structured metadata filters** automatically.

- Filter by title, authors, published\_year, word\_count
- `enable_limit=True` → LLM decides how many

**"...UNDER 50,000 WORDS"**

A pure similarity search can't honor that. The self-query layer translates it into a real filter:

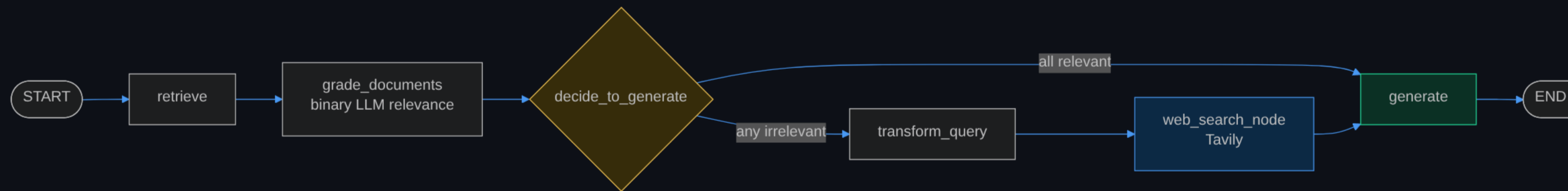
```
word_count < 50000
```

File: `QueryConstruction.py`



## STAGE 4 · THE CORE IDEA

# CORRECTIVE RAG — A LANGGRAPH LOOP



Every retrieved document gets a **yes/no relevance grade**. If anything's off, the graph **rewrites the query** and **falls back to web search** before generating. A real feedback loop — not a one-shot pipeline.

Files: CRAG.py, graphCrag.py

# ANSWER GENERATION

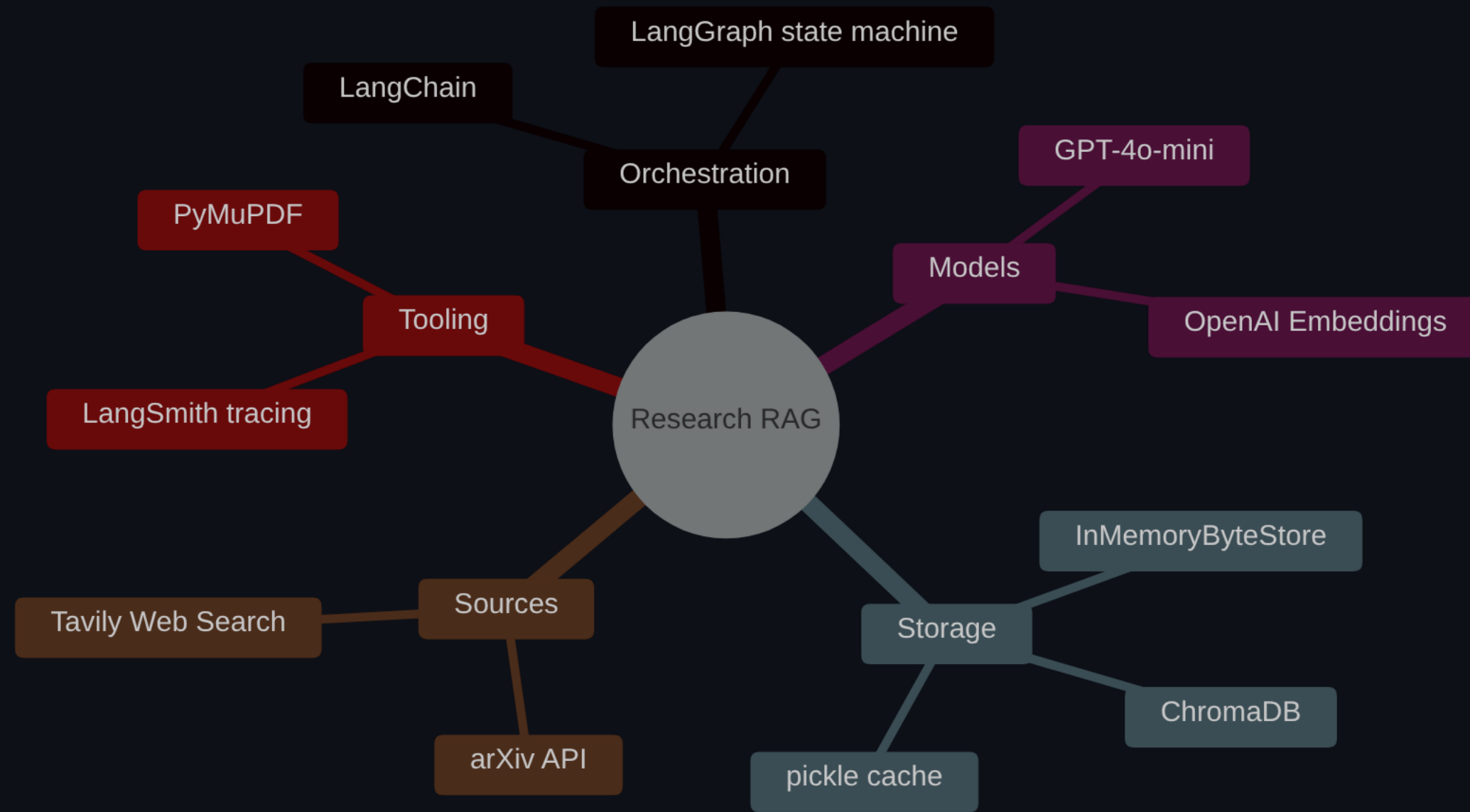
- Final docs + question → `r1m/rag-prompt` from LangChain Hub
- Generated with `GPT-4o-mini`
- Streamed **node-by-node** via `app.stream()` for real-time feedback
- Context kept under a

## GROUNDED BY DESIGN

The answer is built only from documents that **passed grading** — or from fresh web results when the corpus came up short. Far less room to hallucinate.

UNDER THE HOOD

# TECH STACK



Research RAG

END TO END

# A QUERY, START TO FINISH

*"What are the most influential papers on reinforcement learning with words less than 50,000?"*

- 1. Generate 5 variants on "influential RL papers"
- 2. Self-query adds filter `word_count < 50000`
- 3. Retrieve top matches per query, dedup by title
- 4. CRAG grades each doc for RL relevance
- 5. If gaps → rewrite + Tavily web search
- 6. Stream a grounded, cited answer

Research RAG

WHAT I LEARNED

# TAKEAWAYS

## RETRIEVAL > PROMPTS

A grading + correction loop did more for answer quality than any prompt tweak.

## INDEX SUMMARIES

Searching tight summaries and returning full docs beats chunking raw PDFs.

## LET THE LLM FILTER

Self-query unlocks structured questions  
similarity search simply can't answer.

## GRAPHS > CHAINS

LangGraph makes conditional, self-correcting flows clean and observable.

Research RAG

## CACHE AGGRESSIVELY

Pickling docs + summaries turned a costly rebuild into an instant reload.

## ALWAYS HAVE A FALLBACK

Web search means "not in the corpus" no longer means "hallucinate".

THANKS

# KNOW WHEN YOU'RE WRONG.

Research RAG — modular, self-correcting retrieval over academic papers.

LangChain

LangGraph

ChromaDB

GPT-4o-mini

Tavily

★ Code & docs → [github.com/mpeshwe/research-rag](https://github.com/mpeshwe/research-rag)

Inspired by "RAG From Scratch" by LangChain.

